ASE CHAPTER 2

V2 Vocabulary Coverage · Software Life-Cycle Models

Direct study answers for Dr. Ali Fahim's confirmed eight vocabulary blocks

This pack provides concise, rigorous, examinable answers for the eight core vocabulary blocks confirmed directly from **Dr. Ali Fahim** (cohort message, 2026-09-23).

> **Formatting Standard:** - Direct English answers responding immediately to the technical concept. - **Bold text** represents source-grounded definitions, mechanisms, and rules. Plain text represents framing or synthesis. - Concise Arabic notes and exam traps are placed separately outside the English answer. - Exact source locators follow each entry.

Term 01: SDLC, Build & Fix, and Waterfall Model

The Software Development Life Cycle (SDLC) is the sequence of phases through which a software system passes from recognition of a need through development, delivery, and maintenance. Some broader life-cycle descriptions also include retirement. The phases provide a structured way to plan and control development.

Build & Fix is an informal development style in which coding begins after only an initial understanding of the problem; the program is then tested and modified until it works satisfactorily. It can be adequate for very small programs, but for professional or non-trivial software it tends to require sharply increasing effort and produce programs that are difficult to maintain.

The Waterfall Model is a classic linear-sequential life-cycle model in which software development is structured as a cascade of systematic, discrete phases—feasibility study, requirements analysis and specification, design and specification, coding and module testing, integration and system testing, delivery, and maintenance. The output of each preceding phase serves as the frozen, formally documented input to the next, requiring complete requirements upfront and disallowing backward iteration in its pure form.

> **ملاحظة الطالب والفخ الامتحاني:** - Fix & Build قد يصلح لبرنامج صغير جداً، لكنه غالباً ينتج برمجيات مهنية أصعب في التطوير والصيانة كلما كبر حجم المشكلة. - نقطة ضعف الشلال القاتلة هي فرض تجميد المتطلبات مبكراً (Freezing requirements early) وافتراض أن الواقع يسير بتتابع خطي مثالي.

Sources: SDLC & Waterfall: B.B. Agarwal et al. (2010), §2.1–§2.2, pp. 29–41; Build & Fix: Mall (4th ed.), Ch. 1, §1.1.1, pp. 4, 12–13; K.K. Aggarwal & Singh (2007), Slides 4–5. Retirement: K.K. Aggarwal & Singh (2007), Slide 3.

Term 02: Incremental and Iterative Enhancement Models

In incremental delivery, customers identify the system services in outline and prioritize them. The system is divided into releases, each providing part of its functionality; detailed requirements for the first release are defined before it is developed, while later requirements can be analyzed in parallel.

The Iterative Enhancement Model combines linear sequential development with the exploratory feedback of prototyping. The system is partitioned into several releases: the first increment implements a core product addressing basic requirements, while supplementary features are scheduled for subsequent cycles. At the end of each cycle, customer evaluation and operational feedback guide the planning, modification, and feature additions of the next increment.

The terms describe related but distinct emphases in the cited sources: Sommerville's incremental delivery allocates services to releases by priority and details each release's requirements before its development; B.B. Agarwal's Iterative Enhancement begins with a usable core product and uses customer evaluation to guide later increments.

> **ملاحظة الطالب والفخ الامتحاني:** - لا تخلط بين Incremental و Iterative Enhancement: في Incremental Delivery تُحدّد الخدمات وأولوياتها إجمالاً، ثم تُفصّل متطلبات كل إصدار عند تطويره؛ أما Iterative Enhancement فيبدأ بمنتج أساسي قابل للاستخدام ويستخدم تقييم العميل لتخطيط الإضافات التالية. - كلاهما يختلف تماماً عن Iterative Waterfall (الشلال التكراري ذي التغذية الراجعة الداخلية).

Sources: Iterative Enhancement: B.B. Agarwal et al. (2010), §2.6, pp. 47–48; Incremental delivery: Sommerville (9th ed.), §2.3.2, pp. 47–48; K.K. Aggarwal & Singh (2007) Slide 9.

Term 03: RAD and Its Applicability Conditions

Rapid Application Development (RAD) is an incremental life-cycle model that achieves high-speed delivery (typically 60 to 90 days) by dividing system requirements into independent, modular components developed in parallel by multiple specialized teams. Its lifecycle comprises Requirements Planning, User Design, Construction, and Cutover, relying extensively on reusable components, fourth-generation techniques (4GL), and automated visual tools.

RAD is most suitable when the following conditions can be met: 1. **High modularity:** The system requirements and architecture must be easily decomposable into independent, standalone functional units. 2. **Component reusability:** Pre-existing, tested software components or frameworks must be available to assemble rather than code from scratch. 3. **Dedicated user involvement:** End-users and clients must actively participate in rapid-fire Joint Application Development (JAD) sessions and continuous prototype evaluations. 4. **Availability of skilled human resources:** Multiple well-coordinated teams composed of highly skilled, specialized engineers must be available. 5. **Low technical risk and moderate performance demands:** The project must not rely on unproven emerging technologies, and performance must not require intricate low-level system interface tuning.

> **ملاحظة الطالب والفخ الامتحاني:** - مراحل RAD هي (Requirements Planning, User Design, Construction, Cutover)؛ أما توفر المستخدمين أو الكفاءات فهي شروط انطباق (Applicability Conditions) وليست مراحل إضافية. - إذا تعذّر تقسيم النظام إلى أجزاء مستقلة أو كانت التقنية عالية المخاطر، فقد لا يكون RAD مناسباً؛ هذا يضعف ملاءمته ولا يعني فشله حتماً.

Sources: RAD Overview & Phases: B.B. Agarwal et al. (2010), §2.7, pp. 49–50; RAD Applicability Conditions & Activity Names: Mall (4th ed.), §2.3, pp. 100–104; K.K. Aggarwal & Singh (2007) Slides 12–15.

Term 04: Evolutionary and Prototyping Models

The Prototyping Model builds an exploratory, working mock-up of the system during early requirements engineering to help customers and developers clarify vague or ambiguous user needs. In a classic throw-away prototyping process, the prototype is evaluated to refine the Software Requirements Specification (SRS) and is subsequently discarded, after which the actual system is engineered cleanly from structured specifications.

The Evolutionary Development Model is an iterative life-cycle paradigm where development engineering first establishes clear requirements and system scope through repetitive cycles of Analysis-Design-Testing through Prototype-Implementation-Assessment-Evaluation. Crucially, the evolutionary model differs from iterative enhancement because it does not require an externally usable product at the end of each cycle, and requirements are implemented by category rather than by priority.

The fundamental difference is that a classic prototype is a throw-away learning vehicle used primarily to elicit and freeze requirements in the SRS, whereas an evolutionary model uses iterative cycles to evolve the target software architecture directly.

> **ملاحظة الطالب والفخ الامتحاني:** - البروتوتايب الأولي ليس بالضرورة هو المنتج النهائي؛ في الـThrow-away Prototyping يتم إتلاف الكود بعد تثبيت متطلبات الـSRS. - التطوير التطوري (Evolutionary) ينفّذ المتطلبات حسب التصنيف التقني أو استقرار المفاهيم، بينما التحسين التكراري يركّز على أولويات العميل العملية.

Sources: Prototyping & Evolutionary: B.B. Agarwal et al. (2010), §2.3 & §2.5, pp. 41–47; Process Flows: K.K. Aggarwal & Singh (2007) Slides 15–19.

Term 05: The Spiral Model and Risk-Driven Development

The Spiral Model, formulated by Barry Boehm, is an evolutionary metamodel that couples the iterative exploration of prototyping with the controlled, systematic checkpoints of linear sequential models. It is fundamentally driven by explicit project risk management, structuring development into concentric loops rather than fixed linear phases.

The geometry of the spiral model carries precise operational meanings: - **Radial dimension:** Represents the cumulative cost incurred up to the current point in the project. - **Angular dimension:** Represents the progress made along the software development process within each cycle.

Each 360° loop traverses four distinct quadrants (sectors): 1. **Quadrant 1 (Objectives & Alternatives):** Determination of phase objectives, alternative solutions, and project constraints. 2. **Quadrant 2 (Alternative Evaluation & Risk Perceptions):** Evaluation of alternatives based on objectives and project risk perceptions. 3. **Quadrant 3 (Risk Resolution & Engineering Strategies):** Development of strategies to resolve uncertainties and risks (deploying benchmarking, simulation, and prototyping), followed by engineering development. 4. **Quadrant 4 (Next-Cycle Objectives & Review):** Determining objectives to be fulfilled in the next cycle to complete the system, supported by stakeholder evaluation.

> **ملاحظة الطالب والفخ الامتحاني:** - احذر الخلط بين أبعاد الرسم: البعد الشعاعي (Radial) يقيس التكلفة المتراكمة (Cumulative Costs)، بينما البعد الزاوي (Angular) يقيس نسبة الإنجاز والتقدم في الدورة. - يسمى Metamodel لأنه قادر على احتواء أي نموذج آخر في قطاعاته؛ وتذكر أن كتاب أغروال (ص. 45) يربط الربع الثاني بتقييم البدائل حسب المخاطر، والربع الثالث بتنفيذ حلول المخاطر (Prototyping ,Simulation ,Benchmarking)، والرابع بتحديد أهداف الدورة القادمة.

Sources: Spiral Model Mechanics & Quadrants: B.B. Agarwal et al. (2010), §2.4, pp. 44–46; Spiral Geometric Reading Key: K.K. Aggarwal & Singh (2007) Slides 20–21.

Term 06: Unified Process Phases: Inception, Elaboration, Construction, Transition

The Unified Process (UP / RUP) is an iterative, incremental, use-case-driven, and architecture-centric software engineering framework developed by Ivar Jacobson, Grady Booch, and James Rumbaugh. It organizes development along two orthogonal dimensions: horizontal time phases and vertical engineering workflows.

The four sequential phases of the Unified Process are: 1. **Inception:** Establishes the business case, feasibility, basic scope, and vision of the project. **Key milestone: Lifecycle Objectives (LCO), delivering an initial project plan, business model, and high-level use-case survey.** 2. **Elaboration:** Analyzes the problem domain, establishes an executable architectural baseline, and addresses highest-risk elements. **Key milestone: Lifecycle Architecture (LCA), delivering an executable architectural prototype and detailed use-case specifications.** 3. **Construction:** Progressively implements the detailed system components and features based on the validated architecture. **Key milestone: Initial Operational Capability (IOC), delivering a deployable beta product release, user manuals, and completed test suites.** 4. **Transition:** Deploys the system into the operational user environment, conducts user training, beta testing, bug tuning, and acceptance validation. **Key milestone: Product Release, achieving formal customer sign-off and operational handover.**

> **ملاحظة الطالب والفخ الامتحاني:** - ميّز بحزم بين المرحلة (Phase) ومجرى العمل (Workflow): المراحل الأربعة تتبع خط الزمن الأفقي، بينما مجاري العمل (Requirements, Analysis, Design, Implementation, Test) تتكرر بنسب متفاوتة داخل كل مرحلة. - ناتج مرحلة Construction هو Initial Operational Capability (نسخة بيتا)، بينما النسخة النهائية الرسمية (Product Release) تُسلَّم عند نهاية مرحلة Transition.

Sources: Unified Process Phases & Milestones: K.K. Aggarwal & Singh (2007) Slides 23–36; RUP Workflow Distinction: Sommerville (9th ed.), §2.4, pp. 67–69.

Term 07: Selecting a Life-Cycle Model by Requirements and Risk

Model selection is an engineering decision based on aligning project characteristics, organizational constraints, and risk tolerances with the operational assumptions of each SDLC model. There is no universally optimal model; suitability depends on multi-dimensional tradeoffs.

Key evaluation criteria include: 1. **Requirements Stability and Understanding:** - **Well-defined, stable, and completely understood upfront:** Waterfall or Incremental. - **Unclear, evolving, or heavily UI-dependent:** Prototyping or Evolutionary Development. 2. **Project Risk Profile:** - **High technical, schedule, or financial risks:** Spiral Model (explicit risk resolution in each quadrant). - **Low technical risk with standard business workflows:** Waterfall or RAD. 3. **User Involvement Capability:** - **Intensive, continuous customer collaboration available:** RAD or Agile/Prototyping. - **Limited, milestone-only client interaction:** Waterfall or Iterative Enhancement. 4. **Development Team and Delivery Deadlines:** - **Modular architecture, urgent tight schedule (60–90 days), highly skilled staff:** RAD. - **Large, unprecedented system with emerging technology:** Spiral or Evolutionary.

> **ملاحظة الطالب والفخ الامتحاني:** - عوامل الاختيار (Requirements, Risk, Team, User) ليست نماذج برمجية، بل معايير هندسية للمفاضلة. - الشلال يناسب المشاريع قصيرة الأجل ذات المتطلبات المفهومة بالكامل، بينما النموذج الحلزوني مخصص للمشاريع المعقدة ذات المخاطر العالية.

Sources: Model Comparison Table: B.B. Agarwal et al. (2010), §2.8, pp. 50–51 (Table 2.2); Selection Matrices: K.K. Aggarwal & Singh (2007) Slides 36–41.

Term 08: Agile / XP / Scrum

Agile Software Development represents a family of iterative, lightweight life-cycle methodologies founded on the Agile Manifesto principles: prioritizing individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a rigid plan.

Extreme Programming (XP) is an Agile methodology tailored for small-to-medium teams facing rapidly changing requirements. It enforces strict engineering and technical practices including Test-Driven Development (TDD), Pair Programming, Continuous Integration, Refactoring, Small Releases, and on-site customer presence to sustain high code quality.

Scrum organizes development into fixed-length sprint cycles, typically two to four weeks. A customer and development team select features from a product backlog for a sprint; the team organizes its work, holds short daily meetings, and reviews completed work with stakeholders at the end of the sprint. **The Scrum Master supports the process and handles external communication.**

> **ملاحظة الطالب والفخ الامتحاني:** - Scrum يركز على إطار الإدارة وتنسيق الفريق والـSprints (Management Framework)، بينما Extreme Programming (XP) يركز على ممارسات الهندسة البرمجية الدقيقة (كالتطوير الموجه بالاختبار والبرمجة الزوجية). - أجايل ليس نموذجاً أحادياً مجمداً، بل مظلة تضم مناهج تشترك في التكرار والتعاون المستمر مع العميل.

Sources: Agile principles: Sommerville (9th ed.), §3.1, pp. 58–60; XP: §3.3, pp. 65–72; Scrum: §3.4, pp. 73–74.

Retrieval set — Focused Active Recall Drill

1 **How does Agile development respond when requirements change during a project?**

Agile methods develop software in short, iterative increments and involve customers throughout development, so feedback and changing requirements can influence later work. Scrum is one Agile method that organizes work into fixed-length sprints.

Source: Sommerville (9th ed.), §3.1, pp. 58–60.

2 **What is Build and Fix, and why is it unsuitable for most professional projects?**

Build & Fix starts coding after an initial briefing, then repeats testing and fixing until the program works. It may work for a very small program, but as problem size grows, effort and time rise sharply, and the resulting software is difficult to maintain.

Source: Mall (4th ed.), Ch. 1, §1.1.1, pp. 4, 12–13; K.K. Aggarwal & Singh (2007), Slides 4–5.

3 **Distinguish Sommerville's incremental development from B.B. Agarwal's Iterative Enhancement.**

Sommerville's incremental delivery allocates services to releases by priority, defines the first release's detailed requirements before development, and analyzes later requirements in parallel. B.B. Agarwal's Iterative Enhancement starts with a usable core product; customer evaluation of each release helps determine the next increment.

Source: Sommerville (9th ed.), §2.3.2, pp. 47–48; B.B. Agarwal et al. (2010), §2.6, pp. 47–48.

4 **Name four engineering practices associated with XP and state what test-first development means.**

XP practices include small releases, pair programming, continuous integration, and refactoring. In test-first development, tests are written before the code; small code changes are then made and refactored until the tests pass.

Source: Sommerville (9th ed.), §3.3, pp. 65–72.

5 **Describe the main steps of a Scrum sprint and distinguish Scrum from XP.**

The customer and development team select backlog features for a fixed-length sprint; the team organizes the work and meets briefly each day; completed work is reviewed with stakeholders at the sprint's end. Scrum describes the development-management cycle, while XP contributes engineering practices such as test-first development and pair programming.

Source: Sommerville (9th ed.), §3.4, pp. 73–74; XP practices: §3.3, pp. 65–72.